Claude Skills

PDF Processing

Complete Study Notes

Covers: Libraries · Operations · Code Patterns · CLI Tools · Best Practices

Compiled: June 19, 2026

What This Skill Covers
The Claude PDF Skill is a built-in capability that activates whenever a task involves PDF files — reading, creating, merging, splitting, form-filling, watermarking, OCR, or encryption. Claude reads SKILL.md first to apply environment-specific constraints and library choices before writing any code.
Trigger phrases: "PDF", ".pdf", "create a report", "extract tables", "fill this form", "merge documents", "add watermark", "read this PDF"

■ 1. Python Library Overview

The skill uses four main Python libraries. Each has a specific role — use the right tool for the job.

Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'Library' 'frags': [ParaFrag(__tag__='para', bold=1, fontName='Helvetica-Bold', fontSize=9, greek=0, italic=0, link=[], rise=0, text='Library', textColor=Color(1,1,1,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph	Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'License' 'frags': [ParaFrag(__tag__='para', bold=1, fontName='Helvetica-Bold', fontSize=9, greek=0, italic=0, link=[], rise=0, text='License', textColor=Color(1,1,1,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph	Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'Best For' 'frags': [ParaFrag(__tag__='para', bold=1, fontName='Helvetica-Bold', fontSize=9, greek=0, italic=0, link=[], rise=0, text='Best For', textColor=Color(1,1,1,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph	Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'Install' 'frags': [ParaFrag(__tag__='para', bold=1, fontName='Helvetica-Bold', fontSize=9, greek=0, italic=0, link=[], rise=0, text='Install', textColor=Color(1,1,1,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph
pypdf	BSD	Merge, split, rotate, encrypt, metadata	pip install pypdf
pdfplumber	MIT	Text extraction with layout, tables, coords	pip install pdfplumber
reportlab	BSD	Creating PDFs from scratch, styled reports	pip install reportlab
pypdfium2	Apache/BSD	Rendering pages to images, fast text read	pip install pypdfium2

■ **Note:** Always use **--break-system-packages** flag when pip-installing in Claude's environment: pip install reportlab --break-system-packages

■ 2. pypdf — Core File Operations

2.1 Reading a PDF

```
from pypdf import PdfReader, PdfWriter

reader = PdfReader("document.pdf")
print(f"Pages: {len(reader.pages)}")

# Extract all text
text = ""
for page in reader.pages:
    text += page.extract_text()
```

- PdfReader opens any PDF for inspection or extraction.

- reader.pages is a list — index with reader.pages[0] for first page.
- extract_text() is fast but loses layout; use pdfplumber for structured data.

2.2 Merging PDFs

```
writer = PdfWriter()
for pdf_file in ["doc1.pdf", "doc2.pdf", "doc3.pdf"]:
    reader = PdfReader(pdf_file)
    for page in reader.pages:
        writer.add_page(page)

with open("merged.pdf", "wb") as output:
    writer.write(output)
```

✓ **Tip:** Open output file in "wb" (write-binary) mode — always required for PDF output.

2.3 Splitting PDFs

```
reader = PdfReader("input.pdf")
for i, page in enumerate(reader.pages):
    writer = PdfWriter()
    writer.add_page(page)
    with open(f"page_{i+1}.pdf", "wb") as output:
        writer.write(output)
```

2.4 Rotating Pages

```
reader = PdfReader("input.pdf")
writer = PdfWriter()
page = reader.pages[0]
page.rotate(90)  # 90, 180, or 270 degrees clockwise
writer.add_page(page)
with open("rotated.pdf", "wb") as output:
    writer.write(output)
```

2.5 Password Protection

```
# Add password
writer.encrypt("userpassword", "ownerpassword")

# Read encrypted PDF
reader = PdfReader("encrypted.pdf")
if reader.is_encrypted:
    reader.decrypt("password")
```

2.6 Cropping Pages

```
page = reader.pages[0]
page.mediabox.left   = 50   # left margin in points
page.mediabox.bottom = 50   # bottom margin in points
page.mediabox.right  = 550  # right edge
page.mediabox.top    = 750  # top edge
writer.add_page(page)
```

Points = 1/72 inch. A4 page is 595 × 842 points. Letter is 612 × 792 points.

2.7 Watermarking

```
watermark = PdfReader("watermark.pdf").pages[0]
reader = PdfReader("document.pdf")
```

```
writer = PdfWriter()
for page in reader.pages:
    page.merge_page(watermark)  # overlays watermark on each page
    writer.add_page(page)
with open("watermarked.pdf", "wb") as output:
    writer.write(output)
```

■ 3. pdfplumber — Text & Table Extraction

3.1 Basic Text Extraction

```
import pdfplumber

with pdfplumber.open("document.pdf") as pdf:
    for page in pdf.pages:
        text = page.extract_text()
        print(text)
```

- Always use as a context manager (with statement) — closes file handles correctly.
- Preserves layout better than pypdf.extract_text().

3.2 Table Extraction

```
import pdfplumber, pandas as pd

with pdfplumber.open("document.pdf") as pdf:
    all_tables = []
    for page in pdf.pages:
        tables = page.extract_tables()
        for table in tables:
            if table:
                df = pd.DataFrame(table[1:], columns=table[0])
                all_tables.append(df)

combined = pd.concat(all_tables, ignore_index=True)
combined.to_excel("extracted.xlsx", index=False)
```

3.3 Advanced Table Settings

```
table_settings = {
    "vertical_strategy":   "lines",  # or "text", "explicit"
    "horizontal_strategy": "lines",
    "snap_tolerance":      3,
    "intersection_tolerance": 15
}
tables = page.extract_tables(table_settings)
```

✓ **Tip:** Use "text" strategy when PDFs have no visible table borders.

3.4 Text with Coordinates

```
with pdfplumber.open("doc.pdf") as pdf:
    page = pdf.pages[0]
    # All characters with position data
    chars = page.chars
    for char in chars[:5]:
        print(f"{char['text']} at x:{char['x0']:.1f}, y:{char['y0']:.1f}")

    # Extract text within a bounding box (left, top, right, bottom)
    region_text = page.within_bbox((100, 100, 400, 200)).extract_text()
```

3.5 Visual Debugging

```
# Render page as image for visual inspection
img = page.to_image(resolution=150)
img.save("debug_layout.png")
```

■ 4. reportlab — Creating PDFs

4.1 Simple Canvas API

```
from reportlab.lib.pagesizes import letter, A4
from reportlab.pdfgen import canvas

c = canvas.Canvas("output.pdf", pagesize=A4)
width, height = A4   # 595, 842 in points

c.setFont("Helvetica-Bold", 16)
c.drawString(100, height - 100, "Hello World!")
c.line(100, height - 120, 400, height - 120)
c.save()
```

- Canvas coordinates: origin (0,0) is bottom-left. Y increases upward.
- drawString(x, y, text) — x from left edge, y from bottom.

4.2 Platypus (Flow-based) API — Recommended

Platypus handles automatic page breaks, text flow, and complex layouts. Build a "story" list of flowables, then call doc.build(story).

```
from reportlab.lib.pagesizes import A4
from reportlab.platypus import SimpleDocTemplate, Paragraph, Spacer, PageBreak
from reportlab.lib.styles import getSampleStyleSheet

doc = SimpleDocTemplate("report.pdf", pagesize=A4,
    leftMargin=2*cm, rightMargin=2*cm,
    topMargin=2*cm, bottomMargin=2*cm)

styles = getSampleStyleSheet()
story = []

story.append(Paragraph("My Report", styles["Title"]))
story.append(Spacer(1, 12))
story.append(Paragraph("Body text here.", styles["Normal"]))
story.append(PageBreak())

doc.build(story)
```

4.3 Custom Paragraph Styles

```
from reportlab.lib.styles import ParagraphStyle
from reportlab.lib.enums import TA_CENTER, TA_LEFT, TA_RIGHT
from reportlab.lib import colors

my_style = ParagraphStyle(
    "MyStyle",
    fontSize=12,
    leading=18,          # line height
    fontName="Helvetica-Bold",
    textColor=colors.HexColor("#0D1B2A"),
    spaceBefore=8,
    spaceAfter=4,
    alignment=TA_CENTER,
```

```
    leftIndent=10,
    backColor=colors.HexColor("#F0F4FA"),
    borderPad=6
)
story.append(Paragraph("Styled text", my_style))
```

4.4 Tables

```
from reportlab.platypus import Table, TableStyle

data = [
    ["Header 1", "Header 2", "Header 3"],
    ["Row 1",    "Data",     "Value"],
    ["Row 2",    "Data",     "Value"],
]

t = Table(data, colWidths=[5*cm, 5*cm, 5*cm])
t.setStyle(TableStyle([
    ("BACKGROUND",    (0,0), (-1,0), colors.HexColor("#0D1B2A")),
    ("TEXTCOLOR",     (0,0), (-1,0), colors.white),
    ("FONTNAME",      (0,0), (-1,0), "Helvetica-Bold"),
    ("ROWBACKGROUNDS",(0,1), (-1,-1), [colors.HexColor("#F0F4FA"), colors.white]),
    ("GRID",          (0,0), (-1,-1), 0.4, colors.HexColor("#CBD5E1")),
    ("ALIGN",         (0,0), (-1,-1), "CENTER"),
    ("TOPPADDING",    (0,0), (-1,-1), 5),
    ("BOTTOMPADDING", (0,0), (-1,-1), 5),
]))
story.append(t)
```

■ **Warning: Never use Unicode subscript/superscript characters in reportlab. Built-in fonts do not support them — they render as black boxes. Use <sub> and <super> XML tags inside Paragraph text instead.**

4.5 Subscripts & Superscripts

```
# Correct — use XML tags inside Paragraph
Paragraph("H<sub>2</sub>O is water", styles["Normal"])
Paragraph("x<super>2</super> + y<super>2</super> = r<super>2</super>", styles["Normal"])

# WRONG — never use these Unicode chars
# H■O  x²  (will render as black boxes)
```

4.6 Inline HTML in Paragraphs

Paragraph text supports a subset of HTML/XML for formatting:

```
"<b>Bold text</b>"
"<i>Italic text</i>"
"<font color='#1A56C4'>Colored text</font>"
"<font size='14'>Larger text</font>"
"Line 1<br/>Line 2"
"<u>Underlined</u>"
```

■ 5. Command-Line Tools

5.1 pdftotext (poppler-utils)

```
# Basic extraction
pdftotext input.pdf output.txt

# Preserve layout (columns, spacing)
pdftotext -layout input.pdf output.txt

# Specific pages only
pdftotext -f 1 -l 5 input.pdf pages_1_to_5.txt

# With bounding box coordinates (XML output)
pdftotext -bbox-layout document.pdf output.xml
```

✓ **Tip:** pdftotext is the fastest option for plain text extraction from large PDFs.

5.2 pdftoppm — Convert Pages to Images

```
# Convert to PNG at 300 DPI
pdftoppm -png -r 300 document.pdf output_prefix

# Convert pages 1-3 at high res
pdftoppm -png -r 600 -f 1 -l 3 document.pdf hi_res

# Convert to JPEG
pdftoppm -jpeg -jpegopt quality=85 -r 200 document.pdf jpegs
```

5.3 pdfimages — Extract Embedded Images

```
# Extract as JPEG (fastest)
pdfimages -j input.pdf output_prefix

# Extract all formats preserving original
pdfimages -all document.pdf images/img

# Extract with page numbers in filename
pdfimages -j -p document.pdf page_images
```

5.4 qpdf — Repair, Encrypt, Optimize

```
# Merge two PDFs
qpdf --empty --pages file1.pdf file2.pdf -- merged.pdf

# Split pages 1-5
qpdf input.pdf --pages . 1-5 -- pages1-5.pdf

# Rotate page 1 by 90 degrees
qpdf input.pdf output.pdf --rotate=+90:1

# Remove password
qpdf --password=secret --decrypt encrypted.pdf decrypted.pdf

# Optimize / compress
qpdf --optimize-level=all input.pdf compressed.pdf

# Check + repair corrupted PDF
```

```
qpdf --check corrupted.pdf
qpdf --fix-qdf damaged.pdf repaired.pdf
```

■ 6. OCR — Scanned PDFs

Scanned PDFs contain images, not text. Normal extraction returns empty strings. Use pytesseract via pdf2image conversion for OCR.

```
# pip install pytesseract pdf2image --break-system-packages
import pytesseract
from pdf2image import convert_from_path

def extract_text_ocr(pdf_path):
    images = convert_from_path(pdf_path)
    text = ""
    for i, image in enumerate(images):
        text += f"=== Page {i+1} ===\n"
        text += pytesseract.image_to_string(image)
        text += "\n\n"
    return text

result = extract_text_ocr("scanned_doc.pdf")
```

■ **Warning: Tesseract (the underlying OCR engine) must be installed on the system separately. In Claude's container it may already be present — check with: which tesseract**

✓ **Tip:** Use scale=2.0 or higher when rendering pages for OCR — higher resolution improves accuracy.

■ 7. pypdfium2 — Fast Rendering & Text

pypdfium2 wraps Chromium's PDFium library. Faster than pypdf for rendering and text, and a good PyMuPDF alternative.

```
import pypdfium2 as pdfium

# Render page to image
pdf  = pdfium.PdfDocument("document.pdf")
page = pdf[0]
bitmap = page.render(scale=2.0, rotation=0)
img = bitmap.to_pil()
img.save("page_1.png", "PNG")

# Batch render all pages
for i, page in enumerate(pdf):
    bitmap = page.render(scale=1.5)
    bitmap.to_pil().save(f"page_{i+1}.jpg", "JPEG", quality=90)

# Fast text extraction
for i, page in enumerate(pdf):
    text = page.get_text()
    print(f"Page {i+1}: {len(text)} chars")
```

■ 8. JavaScript Libraries

8.1 pdf-lib — Create & Modify PDFs

```
import { PDFDocument, rgb, StandardFonts } from "pdf-lib";
import fs from "fs";

async function createPDF() {
    const pdfDoc = await PDFDocument.create();
```

```
    const font   = await pdfDoc.embedFont(StandardFonts.HelveticaBold);
    const page   = pdfDoc.addPage([595, 842]);  // A4
    const { width, height } = page.getSize();

    page.drawText("Hello from pdf-lib", {
        x: 50, y: height - 80,
        size: 18, font,
        color: rgb(0.1, 0.3, 0.8)
    });

    const bytes = await pdfDoc.save();
    fs.writeFileSync("created.pdf", bytes);
}
```

8.2 pdf-lib — Merge PDFs

```
const merged = await PDFDocument.create();
const pdf1   = await PDFDocument.load(fs.readFileSync("doc1.pdf"));
const pdf2   = await PDFDocument.load(fs.readFileSync("doc2.pdf"));

// Copy all pages from pdf1
const pages1 = await merged.copyPages(pdf1, pdf1.getPageIndices());
pages1.forEach(p => merged.addPage(p));

// Copy specific pages from pdf2 (index 0, 2, 4)
const pages2 = await merged.copyPages(pdf2, [0, 2, 4]);
pages2.forEach(p => merged.addPage(p));

fs.writeFileSync("merged.pdf", await merged.save());
```

8.3 PDF.js (pdfjs-dist) — Browser Rendering

```
import * as pdfjsLib from "pdfjs-dist";

// Always configure worker first
pdfjsLib.GlobalWorkerOptions.workerSrc = "./pdf.worker.js";

const pdf  = await pdfjsLib.getDocument("doc.pdf").promise;
const page = await pdf.getPage(1);
const viewport = page.getViewport({ scale: 1.5 });

const canvas  = document.createElement("canvas");
canvas.width  = viewport.width;
canvas.height = viewport.height;
await page.render({ canvasContext: canvas.getContext("2d"), viewport }).promise;
```

■ 9. Quick Reference — Right Tool for the Job

Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'Task' 'frags': [ParaFrag(__tag__='para', bold=1, fontName='Helvetica-Bold', fontSize=9, greek=0, italic=0, link=[], rise=0, text='Task', textColor=Color(1,1,1,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph	Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'Best Tool' 'frags': [ParaFrag(__tag__='para', bold=1, fontName='Helvetica-Bold', fontSize=9, greek=0, italic=0, link=[], rise=0, text='Best Tool', textColor=Color(1,1,1,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph	Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'Key Method / Command' 'frags': [ParaFrag(__tag__='para', bold=1, fontName='Helvetica-Bold', fontSize=9, greek=0, italic=0, link=[], rise=0, text='Key Method / Command', textColor=Color(1,1,1,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph
Merge PDFs	pypdf	writer.add_page(page)
Split PDFs	pypdf	PdfWriter per page
Rotate pages	pypdf	page.rotate(90)
Add watermark	pypdf	page.merge_page(wm)
Password protect	pypdf	writer.encrypt(pw)
Crop pages	pypdf	page.mediabox.left = N
Extract text (basic)	pypdf	page.extract_text()
Extract text (layout)	pdfplumber	page.extract_text()
Extract tables	pdfplumber	page.extract_tables()
Text with coordinates	pdfplumber	page.chars / within_bbox()
Create from scratch	reportlab	SimpleDocTemplate + Platypus
Styled reports	reportlab	ParagraphStyle + Table
Render to image	pypdfium2	page.render(scale=2.0)
CLI: extract text	pdftotext	pdftotext -layout in.pdf out.txt
CLI: to images	pdftoppm	pdftoppm -png -r 300 in.pdf prefix
CLI: extract images	pdfimages	pdfimages -j in.pdf prefix
CLI: merge	qpdf	qpdf --empty --pages a.pdf b.pdf -- out.pdf
CLI: repair	qpdf	qpdf --fix-qdf broken.pdf fixed.pdf
Scanned PDF text	pytesseract	convert_from_path() + image_to_string()
Fill PDF forms	pdf-lib / pypdf	See FORMS.md

Browser rendering	pdfjs-dist	pdfjsLib.getDocument().promise

■ 10. Best Practices & Gotchas

1.	**Always read SKILL.md first** — Claude reads the skill file before writing any PDF code. Skills encode environment constraints that training data may not reflect.
2.	**Output files go to /mnt/user-data/outputs/** — final deliverables must be placed here for the user to access. /home/claude is a scratch directory only.
3.	**Use "wb" mode for PDF output** — binary write mode is always required: open("output.pdf", "wb").
4.	**Coordinate system in canvas API** — origin is bottom-left, Y increases upward. Platypus API handles this automatically.
5.	**No Unicode sub/superscripts in reportlab** — use <sub> and <super> tags.
6.	**pdfplumber vs pypdf for text** — pdfplumber preserves layout and handles tables; pypdf.extract_text() is faster but loses structure.
7.	**Large PDFs** — process in chunks of 10 pages; use qpdf --split-pages for pre-splitting; use pypdfium2 for memory-efficient rendering.
8.	**Scanned PDFs** — extract_text() returns empty; always fall back to pytesseract OCR pipeline.
9.	**Encrypted PDFs** — check reader.is_encrypted before extracting; call reader.decrypt(pw) first.
10.	**Table extraction fails?** — switch vertical/horizontal strategy from "lines" to "text" for PDFs without visible borders.
11.	**pip install flag** — always use pip install PACKAGE --break-system-packages in Claude's Ubuntu environment.

■ 11. Performance Optimization

Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'Scenario' 'frags': [ParaFrag(__tag__='para', bold=1, fontName='Helvetica-Bold', fontSize=9, greek=0, italic=0, link=[], rise=0, text='Scenario', textColor=Color(1,1,1,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph	**Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'Recommendation' 'frags': [ParaFrag(__tag__='para', bold=1, fontName='Helvetica-Bold', fontSize=9, greek=0, italic=0, link=[], rise=0, text='Recommendation', textColor=Color(1,1,1,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph**
Large PDFs (100+ pages)	Use qpdf --split-pages first; process pages individually with pypdfium2; avoid loading entire file in memory.

Plain text extraction speed	pdftotext -layout is fastest CLI option; pypdf for Python batch; avoid pdfplumber for pure speed.
Structured text / tables	pdfplumber is the only reliable choice; use custom table_settings for complex layouts.
Image extraction from PDF	pdfimages is faster than rendering pages; use -all to preserve original format.
High-quality rendering	pypdfium2 with scale=2.0 for screen quality; scale=3.0+ for print/OCR quality.
Form filling	pdf-lib maintains form field structure best; see FORMS.md for field-by-field filling.
Memory management	Process in chunks of 10 pages; create a new PdfWriter per chunk; write and discard.